programando com python

etapa 3: there and back again: listas e loops

(coleções e estruturas de repetição)

- ➔ listas
- ➔ tuplas
- ➔ dicionários
- ➔ sets
- ➔ operador in
- ➔ loop for
- ➔ loop while
- ➔ comprehensions

coleções

uma coleção é uma estrutura de dados que armazena mais de um valor em uma única variável, permitindo a manipulação de todos esses dados de uma só vez (e também o acesso individual).

Em python temos quatro coleções principais:

- **listas:** coleção de valores ordenados, que podem ser acessados e modificados
- **tuplas:** coleção de valores ordenados, que podem ser acessados mas nunca modificados
- **dicionários:** coleção de valores não ordenados em formato **chave** e **valor**.
- **sets:** coleção de valores não ordenados e sem repetições
- **frozenset():**

listas

em aulas anteriores armazenamos valores em variáveis individuais, ou seja, se temos **três** nomes precisamos de **três** variáveis, com listas podemos criar apenas uma:

```
livros_mistborn = ["O Império Final", "O Poço da Ascensão", "O Herói das Eras"]
```

definimos uma lista em python com o uso de *[].* é uma coleção extremamente versátil e permite armazenar diversos tipos de valores, incluindo tipos diferentes de variáveis, valores repetidos e até mesmo outras listas.

```
frutas_e_numeros = ["limão", "laranja", 334, "abacaxi", 12.4, 0]
```

e acessamos um valor dessa lista usando seu índice (começando com 0):

```
livro1_imperio_final = livros_mistborn[0]
print(f'o livro na posição 0 é: {livro1_imperio_final}')
```

```
o livro na posição 0 é: O Império Final
```

**embora python permita, evite adicionar diferentes tipos dentro de uma mesma lista, é uma prática ruim e que pode causar problemas ao longo da aplicação.*

operações com listas

Também existem diversos tipos de métodos para uma lista em python, facilitando a manipulação dos dados dentro dela:

- .append(var): adicionar a variável no final da lista.
- .extend(lista): estende a lista adicionando elementos de outra lista ao final.
- .insert(pos, var): adiciona a variável na posição solicitada e desloca os outros.
- .pop(pos): retorna o elemento na posição solicitada e o remove da lista. (retorna o último se não passar parâmetro)
- .remove(var): busca e remove a primeira aparição na lista
- .sort(): ordena a lista (esse método modifica a lista original)
- .clear(): limpa completamente a lista
- .index(var): retorna o índice da primeira aparição do valor.
- .count(var): retorna quantas vezes o valor aparece na lista.

mais informações em métodos de listas:
https://www.w3schools.com/python/python_ref_list.asp

operações com listas

algumas funções úteis para listas:

- sorted(lista): retorna uma **nova** lista ordenada (não modifica a original)
- sum(lista): retorna a soma de todos os elementos numéricos da lista
- len(lista): retorna a quantidade de elementos da lista.
- max(lista): retorna o maior valor encontrado na lista.
- min(lista): retorna o menor valor encontrado na lista.

listas e strings são na verdade muito parecidos, sendo a string uma “lista” de caracteres, dessa forma ambos suportam o uso de slicing.

**exemplos no arquivo listas.py*

operações com listas

vamos fazer um teste:

1. crie uma lista chamada *músicas* e atribua 3 valores a ela
2. faça um print dessa lista e verifique seus valores
3. crie uma nova variável e tente atribuir a ela sua lista *musicas* anterior
4. faça um print da nova variável e verifique se deu certo
5. faça uma alteração qualquer na sua nova lista criada a partir da variável, pode adicionar uma nova música, remover uma das que já estão lá ou substituir uma delas
6. faça um novo print nas duas listas existentes
7. o que aconteceu?

operações em listas

isso acontece pois listas em python não armazenam valores diretamente, elas guardam apenas o **endereço na memória** onde os dados estão de fatos armazenados

quando você atribui uma lista em uma variável, o que você está fazendo é passar esses ponteiros já existentes como valor a ser armazenado.

se sua lista original e a nova possuem os **mesmos endereços em memória** armazenados, qualquer **alteração** em uma **vai se refletir** na outra!

duas soluções seriam usar o slicing sem parâmetros de início ou fim para que todos os valores sejam retornados, ou use o método .copy():

```
lista_exemplo = ["A", "B", "C"]

# com slicing
copia_lista = lista_exemplo[:] # sem especificar o início e o fim, pega toda a lista

# com método copy
copia_lista = lista_exemplo.copy() # por baixo dos panos segue a mesma lógica do slicing
```

tuplas

uma tupla é uma lista imutável, dessa forma não pode ser alteradas após a criação.

são úteis para armazenar valores fixos ou sensíveis a alterações, pois garante que os dados permaneçam inalterados durante a execução do programa.

o funcionamento de uma tupla é semelhante ao de uma lista, com exceção do uso de métodos ou funções que realizam alterações dos elementos.

ainda podemos acessar elementos através do índice ou usando slicing, e métodos como count() e funções como len().

podemos declarar uma tupla em python com o uso de *():*

```
coordenadas = (-22.95009428252824, -43.210176394896514)
print(f'coordenadas: {coordenadas}')
```

```
coordenadas: (-22.95009428252824, -43.210176394896514)
```

tuplas de um único elemento precisam de uma vírgula para que python entenda que não se trata de uma precedência de operadores

```
tupla_um_elemento = (42,)
print(f'tupla de um elemento: {tupla_um_elemento}')
```

```
tupla de um elemento: (42,)
```

tuplas - burlando regras 😈

uma tupla é imutável, mas uma lista não.
se a tupla possui uma lista, você consegue de forma simples alterar dados dentro de uma estrutura imutável.

tuplas também são facilmente convertidas para listas com type casting, dessa forma você pode alterar os valores e então fazer o caminho inverso, convertendo sua lista de volta para uma tupla.

tuplas - burlando regras 😈

```
tupla = ("A", "B", "C")
print(f'tupla original: {tupla}')

lista = list(tupla)
lista[0] = "X"
tupla = tuple(lista)

print(f'tupla modificada: {tupla}')
```

```
tupla original: ('A', 'B', 'C')
tupla modificada: ('X', 'B', 'C')
```

converte -> modifica -> converte

```
tupla_com_lista = (["A", "B", "C"],)
tupla_com_lista[0][0] = "X"
print(f'tupla com lista modificada: {tupla_com_lista}')
```

```
tupla com lista modificada: (['X', 'B', 'C'],)
```

alterando valores de dentro

sets (conjuntos)

um set ou conjunto, é uma coleção não ordenada e que não permite duplicatas.

elementos de um set não podem ser acessados através de seu índice, pois sets não são ordenados, e valores já existentes não podem ser adicionados.

a declaração de um set pode ser feita de duas formas:

definindo seus valores no momento da declaração envolvidos em *{}*:

```
numeros = {1,4,5,67,4,334}
print(type(numeros))
print(numeros)
```

```
<class 'set'>
{1, 67, 4, 5, 334}
```

para declarar um set vazio usamos *set()* do contrário um dicionário será instanciado:

```
(variable) conjunto_vazio: set
conjunto_vazio = set()
conjunto_vazio = {}
```

```
(variable) conjunto_vazio: dict
conjunto_vazio = {}
```

sets (conjuntos)

os principais métodos de são:

- .remove(elem) remove o elemento mas causa um erro se não encontrar ele no set
- .discard(elem): remove o elemento ou faz nada se não encontrar ele no set
- .pop(): remove um elemento aleatório do set.
- add(elem): adiciona um elemento no set
- update(lista): adiciona todos os elementos de uma lista ao set

a principal característica de um set é possuir diversas operações matemáticas baseadas na teoria de conjuntos:

- união (| ou .union()): junta todos os elementos de dois conjuntos
- interseção (& ou .intersection()): apenas o que existe em ambos.
- diferença (- ou .difference()): o que tem no primeiro que não tem no segundo
- diferença simétrica (^ ou .symmetric_difference()): tudo o que não é comum aos dois

mais sobre sets: https://www.w3schools.com/python/python_sets.asp

dicionários

um dicionário é uma coleção que utiliza um sistema de ***chave*** e ***valor*** para armazenar os dados, isso faz com que ele seja extremamente eficientes em buscas.

todo elemento de um dicionário é um par, composto por uma chave que precisa ser **imutável (string, int, float, tuple)** e um valor que pode ser de qualquer tipo, as chaves não podem se repetir no dicionário, a última vai sobrescrever a anterior.

usamos chaves para declarar um dicionário em python:

```
pessoa1 = {"nome": "Pedro", "sobrenome": "Vinicius", "idade": 24}
```

e através de uma chave acessamos seu valor:

```
print(pessoa1["nome"])
print(pessoa1["sobrenome"])
print(pessoa1["idade"])
```

```
Pedro
Vinicius
24
```

operações com dicionários

principais métodos:

- **.get(chave, string opcional):** retorna o valor da chave ou None ou a string passada caso a chave não exista, evita erros
- **.keys():** retorna uma lista com todas as chaves
- **.values()**: retorna uma lista com todos os valores
- **.items()**: retorna pares de chave e valor que podem ser iterados em um loop
- .update(dicionario): útil para fundir dois dicionários de uma só vez.
- .pop(chave): remove a chave específica e retorna o valor dela
- .popitem(): remove o último par inserido
- .clear(): limpa o dicionário {}

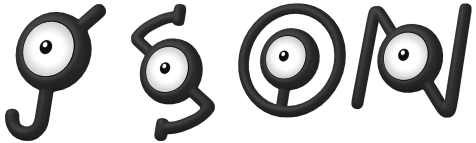
dicionários e json

um dicionário em python é uma implementação da estrutura de dados hash table, ela é extremamente eficiente em buscas e por isso é tão usada. seguindo o conceito de chave valor, temos um tipo de arquivo muito usado para transportar informações entre sistemas, principalmente sistemas web, o .json.

normalmente os dados são transportados em formato json e ao serem recebidos pela aplicação passam por um processo de deserialização para que se encaixem nas estruturas de dados correspondentes.

em python por exemplo, podemos receber um json de uma api e transformar ele facilmente em um dicionário, dessa forma facilitando muito a manipulação dos dados.

**exemplo de código no arquivo pokemon_json.py*



operador in em listas e tuplas (linear)

é um dos operadores mais úteis do python, exemplificando o conceito de simplicidade da linguagem.

com ele podemos verificar a existência (ou não existência) de um item dentro de uma coleção e retorna um booleano (True ou False).

```
paises_brics = ["Brasil", "Rússia", "India", "China", "África do Sul"]

print("Brasil" in paises_brics)
print("Estados Unidos" in paises_brics)
```

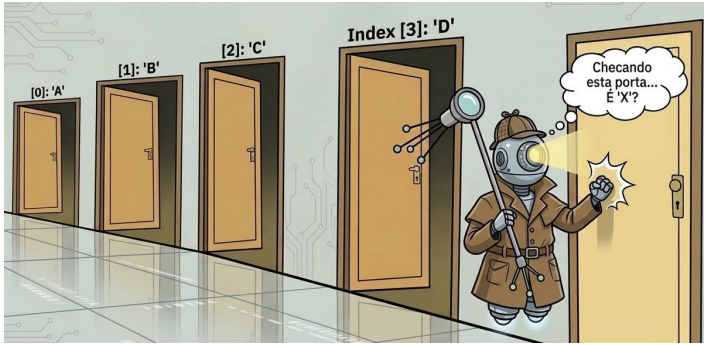
```
True
False
```

seu funcionamento vai variar de acordo com a coleção em que está sendo aplicado, em uma lista ou tupla, ele verifica cada um dos itens até que encontre uma aparição. *O(n)*

```
mega_da_virada_2025 = (59,21,32,13,9,33)

print(59 in mega_da_virada_2025)
print(9 in mega_da_virada_2025)
```

```
True
True
```



operador in em dicionários e conjuntos (hash)

quando usado em um dicionário ou conjunto (set), que como vimos, são estruturas de dados não ordenadas e que implementam o conceito de hash table, o operador in aplica a função de hash no valor e obtém diretamente o endereço em memória, descartando a necessidade de percorrer toda a coleção. *O(1)*

para que isso aconteça, quando efetuamos uma busca em dicionário, olhamos apenas para a existência da **chave**, do contrário seria necessário uma busca linear assim como na lista.

```python
charmander = {"nome": "Charmander", "tipo": "fogo", "nivel": 5}

print("nome" in charmander) # encontra a chave
print("Charmander" in charmander) # nao encontra pois nao e chave
print("Charmander" in charmander.values()) # encontra pois o retorno de .values() é uma "lista"
```

```
True
False
True
```

operador in em strings

o operador também funciona em strings (já que elas são uma lista de caracteres), na verdade sendo até mais poderoso em buscas por string, considerando até mesmo substrings dentro da variável original:

```
string = "o sabor banana caiu como uma luva"
print("banana" in string) # encontra a string
print("ana" in string) # encontra a substring
print("a" in string) # encontra o caractere
print("uva" in string) # encontra a substring na palavra "luva"
print(" uva " in string) # não encontra pois procuramos com espaços
```

```
True
True
True
True
False
```

essa busca não é feita de forma linear, ela usa técnicas como verificar se o caracter na posição final da string é igual ao do valor buscado e então consegue ignorar partes inteiras da iteração, além de não ser necessário obter o ponteiro de memória para verificar valores, já que string são armazenadas em blocos diretamente.

operador not in

também podemos usar o operador **not** em conjunto com o **in**:

```
paises_brics = ["Brasil", "Rússia", "India", "China", "África do Sul"]

pais_buscado = "Suriname"
if pais_buscado not in paises_brics:
    print(f"{pais_buscado} não faz parte dos países do BRICS")
else:
    print(f"{pais_buscado} faz parte dos países do BRICS")
```

```
Suriname não faz parte dos países do BRICS
```

torna o código mais próximo do inglês e facilita o entendimento.

exemplo 1 - barrados no baile

```python
lista_convidados = ("pedro", "tiago", "joão", "maria", "volodmyr zelensky")

nome = input('digite seu nome para verificar entrada: ').lower().strip()

if nome in lista_convidados:
    print(f'{nome} tá na lista, pode entrar!')
else:
    print(f'{nome} não tá na lista, cai fora!')
```

```
digite seu nome para verificar entrada: labubu
labubu não tá na lista, cai fora!
```

```
digite seu nome para verificar entrada: pedro
pedro tá na lista, pode entrar!
```

usando tupla, a lista de convidados será fixa após a criação.

exemplo 2 - adicionando convidados

```python
lista_convidados = ["pedro", "tiago", "joão", "maria", "volodmyr zelensky"]

nome = input('digite nome de quem quer verificar: ').lower().strip()

respostas_sim = ['sim', 's', 'yes', 'y']
respostas_nao = ['não', 'nao', 'n', 'no']

if nome not in lista_convidados:
    print(f'{nome} não tá na lista de convidados')
    print('deseja adicionar a pessoa na lista?')

    resposta = input('responda sim ou não: ').lower().strip()
    if resposta in respostas_sim:
        lista_convidados.append(nome)
        print(f'{nome} adicionado a lista de convidados')
    elif resposta in respostas_nao:
        print(f'{nome} não adicionado a lista de convidados')
    else:
        print('resposta inválida, por favor responda sim ou não')
else:
    print(f'{nome} tá na lista de convidados')
```

```
digite nome de quem quer verificar: clarencio
clarencio não tá na lista de convidados
deseja adicionar a pessoa na lista?
responda sim ou não: s
clarencio adicionado a lista de convidados
lista de convidados atualizada: ['pedro', 'tiago', 'joão', 'maria', 'volodmyr zelensky', 'clarencio']
```

usando lista podemos modificar elementos.

exemplo 3 - consultar estoque

```python
estoque = {
    "banana": 19,
    "laranja": 12,
    "coca-cola": 5,
    "urânio enriquecido": 0,
    "memória RAM DDR5": 1
}

busca = input("qual produto quer veriificar: ").lower().strip()

if busca not in estoque:
    print(f"não vendemos o produto: {busca}")
elif busca in estoque and estoque[busca] > 0:
    print(f"temos {estoque[busca]} unidades de {busca} em estoque. {"corre ta acabando" if estoque[busca] <= 2 else ""}")
else:
    print(f"o produto {busca} está esgotado no momento")
```

```
qual produto quer veriificar: banana
temos 19 unidades de banana em estoque.
```

```
qual produto quer verificar: iphone 27 pro max deluxe director's cut
não vendemos o produto: iphone 27 pro max deluxe director's cut
```

```
qual produto quer verificar: urânio enriquecido
o produto urânio enriquecido está esgotado no momento
```

exemplo 4 - contando estados visitados

```python
viagens_feitas = ["RJ", "SP", "SP", "SP", "RJ", "SP", "MG", "PE", "RJ"]

total_de_viagens = len(viagens_feitas)
print(f"total de viagens: {total_de_viagens}")

estados_visitados = set(viagens_feitas)
total_estados_visitados = len(estados_visitados)
print(f"total de estados visitados: {total_estados_visitados}")

print(f"estados visitados: {estados_visitados}")
```

```
total de viagens: 9
total de estados visitados: 4
estados visitados: {'RJ', 'SP', 'MG', 'PE'}
```

loop for (para)

um loop em python é uma estrutura de repetição, ele permite iterar sobre um objeto sequencial (coleções e strings).

o **loop for** funciona iterando cada um dos elementos definidos inicialmente, definimos o loop usando o operador **for in:**

```
numeros = [1,2,3,4,5,6,7,8,9,10]

for n in numeros:
    print(n)
```

```
1
2
3
4
5
6
7
8
9
10
```

chamamos de **n** cada um dos elementos dentro da lista **numeros**, o loop então acessa cada um desses elementos e realiza a lógica.

loop for

o comportamento do loop vai depender da coleção em que está sendo usado:

- listas e tuplas: segue a ordem crescente de acordo com os índices, iniciando em 0.
- conjuntos (sets): itera sobre cada um dos elementos de forma aleatória, devido a natureza de um set (não ordenado).
- dicionários: itera somente sob as chaves do dicionário.

```
country = {"nome": "Brasil", "capital": "Brasília", "populacao": 213000000}
for chave in country:
    print(chave)
```

```
nome
capital
populacao
```

loop for - ferramentas úteis

podemos usar algumas funcionalidade no python para melhorar loops do tipo for:

- **enumerate():** pode ser usado quando você precisa não só do item iterado mas também da sua posição na coleção, retorna o **index** e o **item**.

```python
frutas = ["banana", "maçã", "laranja", "uva", "abacaxi"]

for i, fruta in enumerate(frutas):
    print(f"{i}: {fruta}")
```

```
0: banana
1: maçã
2: laranja
3: uva
4: abacaxi
```

loop for - ferramentas úteis

- **range(int,):** pode ser usado para criar loops for sem ter necessariamente uma coleção para percorrer, segue a lógica de parâmetros do **slicing** *(começo, fim, passo)*

```python
print("\n de 0 a 4")
for i in range(5):
    print(i)

print("\n de 0 a 9 de 2 em 2")
for i in range(0, 10, 2):
    print(i)

print("\n de 1 a 9")
for i in range(1, 10):
    print(i)
```

```
 de 0 a 4
0
1
2
3
4

 de 0 a 9 de 2 em 2
0
2
4
6
8

 de 1 a 9
1
2
3
4
5
6
7
8
9
```

```python
print("\n iterando sobre os indices da lista frutas")
for i in range(len(frutas)):
    print(f"{i}: {frutas[i]}")
```

```
 iterando sobre os indices da lista frutas
0: banana
1: maçã
2: laranja
3: uva
4: abacaxi
```

loop for - ferramentas úteis

- **zip():** percorre duas listas ao mesmo tempo, buscando os elementos de cada uma delas:

```python
print("\n iterando sobre lista1 - letras e lista2 - numeros")
lista1 = ["a", "b", "c"]
lista2 = [3,2,1]

for letra, numero in zip(lista1, lista2):
    print(f"{letra}: {numero}")
```

```
 iterando sobre lista1 - letras e lista2 - numeros
a: 3
b: 2
c: 1
```

loop for - ferramentas úteis

- **.items():** extremamente útil para iterar elementos de um dicionário, já que o loop percorre por padrão apenas as chaves, .items() retorna uma tupla com par dos valores:

```
games_mais_vendidos = {"PSP": "GTA: Liberty City Stories",
                       "PS2": "GTA: San Andreas",
                       "NDS": "New Super Mario Bros",
                       "3DS": "Mario Kart 7"}

for console, jogo in games_mais_vendidos.items():
    print(f"{console}: {jogo}")
```

```
PSP: GTA: Liberty City Stories
PS2: GTA: San Andreas
NDS: New Super Mario Bros
3DS: Mario Kart 7
```

loop for - ferramentas úteis

- **else:** se adicionado após qualquer loop em python, a condicional else será executada ao final das iterações / repetições:

```python
numeros = [1,2,3]

for n in numeros:
    print(n)
else:
    print("Fim do loop.")
```

```
1
2
3
Fim do loop.
```

também funciona no loop while.

loop for aninhado

loops podem ser aninhados, é uma forma de acessar por exemplo listas dentro de listas (matrizes):

```
matriz = [[65, 212], [1113, 42]]

for linha in matriz:
    for item in linha:
        print(item)
```

```
65
212
1113
42
```

loop while

diferente do **loop for**, o while depende de uma condição / estado para interromper a execução, a condição deve ser atingida para que o loop finalize:

```
contador = 0

while contador < 10:
    print(f"contando: {contador}")
    contador+=1
```

```
contando: 0
contando: 1
contando: 2
contando: 3
contando: 4
contando: 5
contando: 6
contando: 7
contando: 8
contando: 9
```

a condição é que a variável contador se torne maior que 10

se a condição nunca for atingida, o loop executa pra sempre!

```
# loop infinito
while True:
    print("loop infinito")
```

loop while - while true

é comum em python usar um **loop while** com a **condição True** e um **break** para encerrar o loop quando necessário, podemos usar esse truque quando não sabemos exatamente até quando vamos precisar iterar:

```
while True:
    algo = input("digite algo: ")
    if algo == "sair":
        print("saindo do loop")
        break
```

```
digite algo: a
digite algo: abc
digite algo: sair
saindo do loop
```

loop while - while collection

podemos usar um while em cima de uma coleção se quisermos garantir que o loop contine até que ela esteja vazia, como vimos antes, uma coleção vazia equivale a False para python, enquanto ela possuir elementos será uma condicional True:

```python
while lista_de_tarefas:
    lista_de_tarefas.pop(0)
    print(f"tarefa finalizada, tarefas restantes: {len(lista_de_tarefas)}")
else:
    print("todas as tarefas foram finalizadas")
```

```
tarefa finalizada, tarefas restantes: 3
tarefa finalizada, tarefas restantes: 2
tarefa finalizada, tarefas restantes: 1
tarefa finalizada, tarefas restantes: 0
todas as tarefas foram finalizadas
```

controle de loop

podemos usar algumas palavras chave para manipular um loop em python:

- **continue:** interrompe a iteração atual e passa para a próxima, em um loop for pulamos para o próximo elemento, já no while voltamos ao início do loop.
- **break:** interrompe o loop, se usado em loops aninhados, retorna ao loop externo.
- **pass:** é um placeholder, não executa nenhuma ação, útil quando ainda não temos o código com a lógica pronto.

comprehensions

listas e dicionários possuem uma forma de declaração simplificada extremamente poderosa chamada **comprehension**.

com ela podemos criar uma dessas coleções usando lógica complexa em apenas **uma linha**, pense nela como um **loop compactado**.

além de ser mais eficiente, pois em um comprehension a coleção é criada no momento da interpretação pelo código em linguagem C.

list comprehension

um comprehension segue uma sintaxe simples e que precisa ser respeitada:

[expressão for item in coleção if condição]

- **expressão:** o que será adicionado na lista (o resultado).
- **for item em coleção**: o loop que vimos ainda agora
- **if condição**: (opcional) filtro para decidir quem entra na lista

list comprehension

```python
# com loop
numeros = []
for i in range(50):
    numeros.append(i)
print(f"com loop: {numeros}")

#comprehension
numeros_comprehension = [i for i in range(50)]
```

```
----------------------------------------
com loop: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
com comprehension: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
```

list comprehension

```python
# usando loop
num_pares = []
for i in range(10):
    if i % 2 == 0:
        num_pares.append(i)
print(f"com loop: {num_pares}")

#comprehension
num_pares_comprehension = [i for i in range ( 10) if i % 2 == 0]
print(f"com comprehension: {num_pares_comprehension}")
```

```
com loop: [0, 2, 4, 6, 8]
com comprehension: [0, 2, 4, 6, 8]
```

list comprehension

```python
#loop
frutas = ["abacaxi", "banana", "acerola", "uva", "ameixa", "melancia"]
frutas_com_a= []

for x in frutas:
    if x.startswith("a"):
        frutas_com_a.append(x)
print(f"com loop: {frutas_com_a}")

#comprehension
frutas_com_a_comprehension = [x for x in frutas if x.startswith("a")]
print(f"com comprehension: {frutas_com_a_comprehension}")
```

```
com loop: ['abacaxi', 'acerola', 'ameixa']
com comprehension: ['abacaxi', 'acerola', 'ameixa']
```

dictionary comprehension

podemos aplicar a mesma lógica para criar **dicionários** a partir de um **comprehension**, basta adaptar a **sintaxe**:

{ chave: valor for item in coleção }

as principais aplicações de um dictionary comprehension são as transformação de dados.

podemos utilizar valores de uma lista para popular um dicionário, ou manipular valores originais de um dicionário para outro.

dictionary comprehension

```python
# dicionário com os quadrados dos números

# usando loop
quadrados_loop = {}
for x in range(6):
    quadrados_loop[x] = x**2
print(f"com loop: {quadrados_loop}")

# comprehension
quadrados_comprehension = {x: x**2 for x in range(6)}
print(f"com comprehension: {quadrados_comprehension}")
```

```
com loop: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
com comprehension: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

dictionary comprehension

```python
# filtrando dicionario já existente, apenas alunos com nota na media ou acima

# loop
alunos_notas = {"Ana": 8, "João": 5, "Maria": 9, "Pedro": 6, "Lucas": 7}
aprovados_loop = {}
media = 7

for aluno, nota in alunos_notas.items():
    if nota >= media:
        aprovados_loop[aluno] = nota
print(f"com loop: {aprovados_loop}")

# comprehension
aprovados_comprehension = {aluno: nota for aluno, nota in alunos_notas.items() if nota >= media}
print(f"com comprehension: {aprovados_comprehension}")
```

```
----------------------------------------
com loop: {'Ana': 8, 'Maria': 9, 'Lucas': 7}
com comprehension: {'Ana': 8, 'Maria': 9, 'Lucas': 7}
----------------------------------------
```

dictionary comprehension

```python
# com loop
palavras = ["paralelepipedo", "abacaxi", "aeronave", "velocidade", "computador"]
tamanho_palavras_loop = {}
for palavra in palavras:
    tamanho_palavras_loop[palavra] = len(palavra)
print(f"com loop: {tamanho_palavras_loop}")

# comprehension
tamanho_palavras_comprehension = {palavra: len(palavra) for palavra in palavras}
print(f"com comprehension: {tamanho_palavras_comprehension}")
```

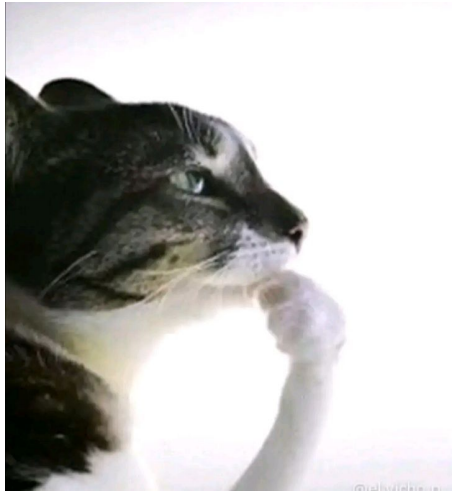
```
----------------------------------------
com loop: {'paralelepipedo': 14, 'abacaxi': 7, 'aeronave': 8, 'velocidade': 10, 'computador': 10}
com comprehension: {'paralelepipedo': 14, 'abacaxi': 7, 'aeronave': 8, 'velocidade': 10, 'computador': 10}
```

exercicios

camarão que dorme a onda leva

a link to the past

antes de fazer os exercicios dessa aula, vale a pena dar uma olhada nos anteriores, coleções e loops são muito úteis e provavelmente vão permitir resolver problemas antigos de forma mais fácil



exercicio 1 - cor existe?

crie uma lista de cores, receba do usuário uma cor e verifique se a cor está presente na sua lista original, exiba uma mensagem de acordo com o resultado.

exercicio 2 - imc here we go again

inicie um programa que pede ao usuário seu peso e altura, ambos devem ser do tipo float, se o valor inserido não for do tipo float, o programa deve continuar solicitando que o usuário insira o valor.

ao final exiba o resultado do imc a partir dos valores.

exercicio 3 - média da turma de n alunos

crie um programa que solicita ao usuário a quantidade de notas a serem inseridas (cada aluno tem apenas 1 nota), receba os valores um de cada vez, ao final exiba:

a maior nota da turma.

a menor nota da turma.

a média das notas da turma.

exercicio 4 - pede pra sair

crie um programa que **solicita** do usuário um **input** qualquer, o programa deve **continuar solicitando** o input até que a palavra “**beterraba**” seja inserida.

exercicio 5 - conjunto vazio

ao adicionar um elemento ao conjunto declarado vazio no código abaixo, o código resulta em erro, corrija o problema:

```
meu_conjunto = {}

meu_conjunto.add(123)

print(meu_conjunto)
```

exercicio 6 - pede pra sair

crie um programa que **solicita** do usuário um **input** qualquer, o programa deve **continuar solicitando** o input até que a palavra “**beterraba**” seja inserida.

exercicio 7 - contador de vogais

crie um programa que recebe uma string qualquer do usuário e conta a quantidade de vogais presentes nessa string, exiba:

a quantidade de vogais.

a quantidade de consoantes.

exercicio 8 - preços

crie um **dicionário** chamado produtos, **iniciado vazio**, em seguida, considerando **chave sendo nome** e **valor sendo o preço**, **adicione cinco** produtos no dicionário.

exiba o preço do **terceiro** produto do dicionário.

exercicio 9 - lista de quadrados

crie uma lista de números inteiros aleatórios, em seguida crie uma nova lista composta pelos quadrados da lista original.

exibia ambas as listas ao final.

exercicio 10 - frequencia de termos

crie um programa que ao receber uma string, faça a contagem de quantas vezes uma letra aparece, armazene essa informação em formato de dicionário.

exiba o dicionário criado.

considere apenas strings sem acentuação!

ex: “Ana Banana me ama”

{'a': 7, 'n': 3, 'b': 1, 'm': 2, 'e': 1}

exercicio 11 - juntando dados

declare 3 listas:

horario = ["10:10", "11:10", "20:30"]

temperatura = [22.4, 37.1, 28.0]

humidade = [76, 44, 66]

use zip() para criar medições baseadas na posição de cada elemento, cada medição é representada por um dicionário e adicionado em uma lista.

exibia a lista ao final.

exercicio 12 - mini biblioteca

crie um sistema de biblioteca, cada livro deve ser representado por um dicionário contendo chaves como nome, isbn, qtd_paginas, os livros deverão ser armazenados em uma lista.

usando um loop, crie um menu que permite ao usuário selecionar entre:
1 - ver livros: deve exibir a lista de livros

2 - add livro: deve adicionar um novo livro na lista

3 - remover livro: deve remover um livro usando seu nome

4 - sair: encerra o programa.